

TRIO GRUPO FINANCEIRO

Infraestrutura de Pagamentos & Tecnologia Financeira

DESAFIO TÉCNICO

Engenheiro de Dados Sênior

Prazo:	7 dias corridos
Desafios:	3 blocos técnicos
Entrega:	Código + documentação + apresentação
Classificação:	CONFIDENCIAL

1. Contexto e Visão Geral

A Trio é um grupo financeiro brasileiro fundado em 2020, especializado em infraestrutura de pagamentos de alto volume e e-banking para empresas. Instituição de Pagamento autorizada pelo Banco Central e participante direta do Pix, a Trio processa grandes volumes de transações em tempo real com tecnologia 100% proprietária. O Grupo destaca-se por sua presença nos segmentos de pagamentos corporativos, iGaming e serviços financeiros integrados.

A plataforma de dados da Trio sustenta toda essa operação e está estruturada em três pilares:

- TimescaleDB (Timescale Cloud): banco transacional principal. Armazena transações financeiras, eventos de pagamento, séries temporais de alta volumetria e dados de reconciliação. É o coração operacional.
- ClickHouse: motor de dados para aplicações e consultas em geral — desde dashboards analíticos (Grafana, Hex) usados pelos Data Champions (analistas distribuídos nos times comercial, financeiro e marketing) até consultas consumidas diretamente por aplicações e serviços internos.
- PostgreSQL (legado): banco transacional legado que pode sustentar parte dos microsserviços. Pode ser migrado para Aurora/RDS na AWS conforme a evolução da stack. O Engenheiro de Dados precisa saber operar, otimizar e eventualmente planejar migrações desse legado.

A infraestrutura roda sobre AWS, e o Engenheiro de Dados precisa transitar com fluência entre Timescale Cloud (managed), ClickHouse, PostgreSQL/Aurora e os serviços AWS que suportam a operação (EC2, RDS, S3, VPC, IAM, CloudWatch, etc).

Este desafio simula cenários reais do dia a dia. O ambiente é 100% autocontido via docker-compose.

2. Instruções Gerais

2.1 Prazo e Entrega

- Prazo: 7 dias corridos a partir do recebimento.
- Entregue um repositório Git (GitHub, GitLab ou zip) contendo todo o código, scripts, configurações e documentação.
- Inclua um README.md raiz com instruções claras de setup e execução.

- Após a entrega, haverá uma apresentação técnica ao vivo (30–45 min) onde você defenderá suas decisões, demonstrará o ambiente rodando e responderá a perguntas técnicas.

2.2 Ambiente Fornecido

Você receberá junto a este documento um arquivo docker-compose.yml base contendo:

- TimescaleDB 2.x (porta 5432) — banco transacional principal
- PostgreSQL 16 (porta 5433) — legado transacional
- ClickHouse latest (portas 8123/9000) — motor de dados para aplicações e consultas
- Grafana (porta 3000) — dashboards

Você deve evoluir este docker-compose conforme necessário (adicionar volumes, scripts de init, serviços extras, etc). Sinta-se livre para adicionar ferramentas como Kafka, Debezium, scripts Python ou qualquer componente que julgue relevante.

2.3 O que Avaliamos

Categoria	Peso	O que observamos
Profundidade Técnica	40%	Domínio real de TimescaleDB, ClickHouse e PostgreSQL. Tuning, modelagem, otimização. Visão de AWS.
Arquitetura e Visão	25%	Capacidade de desenhar soluções escaláveis em cloud, justificar trade-offs, pensar em evolução e migrações.
Operação e Resiliência	20%	Backup, recovery, observabilidade, resposta a incidentes, segurança operacional em ambiente AWS.
Comunicação e Liderança	15%	Clareza na documentação, capacidade de explicar decisões, postura de liderança técnica junto aos Data Champions.

3. Desafio 1 — Performance e Tuning de Queries

Tempo sugerido: 2 a 3 dias

3.1 Cenário

A Trio processa milhões de transações financeiras diárias entre Pix, TEDs, boletos e cartões. Os dados transacionais vivem no TimescaleDB, parte do legado permanece no PostgreSQL, e o ClickHouse alimenta aplicações, dashboards e consultas dos Data Champions. Você receberá scripts de geração de dados sintéticos — ou deve criá-los — para simular carga real.

3.2 Tarefas

PARTE A — TimescaleDB: Modelagem Transacional e Séries Temporais

1. Crie o schema transacional no TimescaleDB com as seguintes tabelas (adapte conforme julgar necessário):
 - transactions: id, external_id (UUID), amount (numeric), currency, status (pending/settled/failed/reversed), type (pix/ted/boleto/card), source_institution, destination_institution, created_at, settled_at, metadata (jsonb).
 - accounts: id, account_number, institution_code, holder_document (cpf/cnpj), holder_name, account_type, status, created_at.
 - reconciliation_events: id, transaction_id, event_type, external_reference, amount_expected, amount_received, difference, reconciled_at, notes.
2. Configure transactions como hypertable. Defina chunk interval, dimensão de particionamento temporal, e justifique suas escolhas considerando o volume e os padrões de consulta.
3. Gere pelo menos 10 milhões de transações distribuídas em 12 meses, com distribuição realista (mais Pix que boletos, picos em dias úteis, variação de status, múltiplas instituições).
4. Implemente Continuous Aggregates para:
 - a) Volume e valor total por tipo de transação, por hora.
 - b) P95 e P99 de latência de liquidação (settled_at - created_at) por instituição, por dia.
5. Configure políticas de retenção e compressão:

CONFIDENCIAL

- Dados granulares (raw) mantidos por 90 dias.
 - Continuous aggregates mantidos por 2 anos.
 - Compressão automática de chunks com mais de 7 dias.
6. Escreva e optimize as seguintes queries, documentando EXPLAIN ANALYZE antes e depois de cada otimização:
 - a) **Q1:** Volume e valor total de transações por tipo e status, agrupado por mês, nos últimos 6 meses.
 - b) **Q2:** Transações com divergência na reconciliação (amount_expected vs amount_received) acima de R\$0,01 nos últimos 30 dias, com dados das contas de origem e destino.
 - c) **Q3:** Top 20 instituições por volume transacionado nos últimos 90 dias, com média de tempo de liquidação e taxa de falha.
 - d) **Q4:** Detecção de possíveis transações duplicadas (mesmo valor, mesma origem, mesmo destino, janela de 5 minutos) — padrão comum em prevenção a fraudes.
 7. Para cada query: índices criados (justificando), impacto do particionamento temporal, EXPLAIN ANALYZE comparativo, e comentários sobre o plano de execução.
 8. Escreva uma query usando funções específicas do TimescaleDB (time_bucket, time_bucket_gapfill, locf/interpolate) para gerar série temporal contínua de volume transacional por hora nas últimas 48h, preenchendo lacunas.
 9. Documente como faria a sanitização de chunks comprimidos que contenham dados com PII (ex: solicitação LGPD de exclusão). Considere o impacto da compressão nesse processo.

PARTE B — PostgreSQL Legado: Avaliação e Visão de Migração

1. Crie um schema simplificado no PostgreSQL representando o legado (tabelas de usuários, contas e parâmetros de configuração de instituições parceiras). Popule com dados sintéticos.
2. Escreva pelo menos 2 queries complexas nesse legado (ex: relatório de contas ativas por instituição com agregações; busca de configurações com joins em múltiplas tabelas) e optimize com índices e ajustes, documentando EXPLAIN ANALYZE.
3. Escreva um documento de 1 página com sua análise e recomendação para evolução do legado PostgreSQL. Considere:

- Manter PostgreSQL on-premise/EC2 vs migrar para Aurora PostgreSQL vs RDS.
- Critérios de decisão: custo, performance, alta disponibilidade, operational overhead.
- Estratégia de migração com zero/mínimo downtime (DMS, replicação lógica, blue-green, etc).
- Riscos e plano de rollback.

PARTE C — ClickHouse: Motor de Dados para Aplicações e Consultas

1. Projete o schema no ClickHouse para receber dados transacionais e servir tanto aplicações internas quanto consultas dos Data Champions. Justifique:
 - Engine escolhida (MergeTree, ReplacingMergeTree, AggregatingMergeTree, etc) e por quê.
 - Chave de ordenação (ORDER BY) — como ela impacta os padrões de consulta das aplicações e dos Data Champions.
 - Estratégia de particionamento (PARTITION BY).
 - Settings de compressão por coluna, se aplicável.
 2. Crie Materialized Views no ClickHouse para pré-agregar:
 - a) Resumo diário por instituição e tipo de transação (contagem, soma, média, p95 de valor).
 - b) Funil de status de transações (pending → settled/failed/reversed) com tempo médio em cada estágio.
 3. Implemente pelo menos um Dictionary (ex: institution_code → institution_name, ou currency → currency_details) e demonstre seu uso em uma query. Explique quando e por que Dictionaries são preferíveis a JOINS no ClickHouse.
 4. Escreva uma query complexa para uso em Grafana: “Taxa de sucesso de transações Pix por instituição por hora nas últimas 24h, com comparação ao mesmo horário do dia anterior (delta %)”. Otimize para rodar em sub-segundo mesmo com centenas de milhões de registros.
 5. Demonstre um cenário onde o ClickHouse serve diretamente uma aplicação (não apenas dashboards): por exemplo, uma API que consulta volume transacional em tempo real para exibição em um painel de operações ou para alimentar regras de negócio. Pode ser um script/endpoint simples que consulta o ClickHouse e retorna JSON.
-

4. Desafio 2 — Pipelines de Dados e Arquitetura

Tempo sugerido: 1 a 2 dias

4.1 Cenário

Os dados precisam fluir do TimescaleDB transacional para o ClickHouse de forma confiável, com baixa latência e garantias de consistência. Dados do legado PostgreSQL também precisam ser orquestrados. Os Data Champions dependem de dados frescos para dashboards e aplicações consomem o ClickHouse diretamente.

4.2 Tarefas

PARTE A — Pipeline TimescaleDB → ClickHouse

1. Projete e implemente o pipeline principal que sincroniza transações do TimescaleDB para o ClickHouse. Você pode escolher a abordagem:
 - CDC (Change Data Capture) com Debezium/Kafka.
 - Replicação lógica nativa do PostgreSQL (base do TimescaleDB).
 - Pipeline custom (script Python, cronjob, micro-batches).
 - Outra abordagem que julgue adequada.

Independente da escolha, justifique por escrito: por que essa abordagem e não as outras? Quais trade-offs?

2. O pipeline deve:
 - Ser idempotente (re-execução não gera duplicatas).
 - Lidar com falhas (retry, dead-letter, logging).
 - Ter observabilidade mínima (logs estruturados, métricas de lag/throughput).
 - Ser demonstrável no docker-compose (rode e funcione).
3. Implemente tratamento para mutações: quando uma transação muda de status (ex: pending → settled), como você garante que o ClickHouse reflete essa mudança corretamente? Demonstre na prática.

PARTE B — Orquestração do Legado PostgreSQL

1. Implemente um pipeline secundário que sincronize dados de referência do PostgreSQL legado (instituições, contas, configurações) para o ClickHouse,

alimentando os Dictionaries criados no Desafio 1. Considere frequência de atualização e consistência.

2. Descreva como esse pipeline se comportaria em um cenário de migração do PostgreSQL para Aurora: o que mudaria? O que precisaria ser adaptado? O pipeline sobreviveria à migração sem alterações?

PARTE C — Desenho Arquitetural e ADR

1. Produza um diagrama arquitetural mostrando o fluxo completo de dados desde a origem até o consumo final. Inclua:
 - Todos os componentes: TimescaleDB (Timescale Cloud), PostgreSQL/Aurora (legado), ClickHouse, pipelines, filas (se houver), Grafana, Hex, aplicações consumidoras.
 - Serviços AWS envolvidos (VPC, subnets, S3 para backups, CloudWatch, etc).
 - Direção dos fluxos e SLAs esperados (latência, freshness).
 - Pontos de falha e estratégias de mitigação.
2. Escreva um Architecture Decision Record (ADR) de 1–2 páginas justificando as escolhas. Considere: por que não um ETL tradicional? Como escalar se o volume 10x? Onde entra o legado PostgreSQL/Aurora nessa evolução? Quais serviços AWS você alavancaria para resiliência e escala?

5. Desafio 3 — Operação, Resiliência e Observabilidade

Tempo sugerido: 1 a 2 dias

5.1 Cenário

Em infraestrutura de pagamentos de missão crítica, incidentes acontecem. A capacidade de operar, diagnosticar e recuperar ambientes de dados sob pressão é tão importante quanto construir. A Trio processa Pix em tempo real para milhares de empresas — qualquer minuto de indisponibilidade tem impacto direto. Este desafio avalia sua maturidade operacional.

5.2 Tarefas

PARTE A — Backup, Recovery e Continuidade

1. Implemente uma estratégia de backup para os três bancos (TimescaleDB, PostgreSQL, ClickHouse) no docker-compose. Para cada um, defina e implemente:
 - Tipo de backup (físico/lógico, full/incremental).
 - Frequência e retenção.
 - Script/automação funcional que execute o backup.
 - Onde armazenaria em produção na AWS (S3, lifecycle policies, cross-region, etc).
2. Demonstre um procedimento de recovery: simule a perda de dados (DELETE de transações em um período) e restaure a partir do backup. Documente o passo a passo com timestamps e validações (contagem antes, depois da perda, e após recovery).
3. Escreva um runbook para o seguinte cenário crítico:

CENÁRIO: O Timescale Cloud está com storage em 92% de utilização. Chunks comprimidos de 6+ meses precisam ser sanitizados e removidos sem downtime e sem perder dados agregados nos continuous aggregates.

O runbook deve incluir: pré-requisitos e validações antes de começar, passo a passo com comandos SQL, checkpoints de validação, plano de rollback, e comunicação para stakeholders.

PARTE B — Observabilidade e Monitoramento

1. Configure dashboards no Grafana com:
 - c) **Dashboard TimescaleDB:** conexões ativas, queries lentas, saúde dos chunks, jobs de compressão/retenção, tamanho por hypertable, status dos continuous aggregates.
 - d) **Dashboard ClickHouse:** queries em execução, memória consumida, merges em andamento, inserções/segundo, tamanho de partições, latency de queries das aplicações.
 - e) **Dashboard de Pipeline:** lag de sincronização, throughput de eventos, erros, última execução com sucesso.
 - f) **Dashboard PostgreSQL Legado:** conexões, queries lentas (pg_stat_statements), bloat estimado. Inclua métricas que ajudariam a justificar a migração para Aurora.

2. Defina pelo menos 5 alertas críticos que você configuraria em produção. Para cada um: métrica, threshold, severidade, ação esperada, e como integraria com CloudWatch/SNS na AWS.

PARTE C — Cenário de Incidente (War Room Simulado)

Responda por escrito, de forma detalhada, como investigaria e resolveria o seguinte incidente:

INCIDENTE SEV-1 — 06h47 (horário de Brasília)

Alerta: Dashboards de transações Pix no Grafana (ClickHouse) mostram volume ZERO nas últimas 2 horas. Aplicações que consultam o ClickHouse estão retornando dados desatualizados. O time de operações confirma que transações estão sendo processadas normalmente no TimescaleDB.

Impacto: Time comercial sem visibilidade de volume em tempo real. Time financeiro sem dados para reconciliação intraday. Painel de operações (aplicação que consome ClickHouse) mostrando dados stale. Head de Cloud/Data acionado.

Contexto adicional: Na noite anterior, houve manutenção programada no TimescaleDB para compressão de chunks antigos. Também foi aplicada uma atualização de security group na VPC da AWS.

Na sua resposta, inclua:

- Linha de investigação (o que verificaria, em que ordem, quais comandos/queries usaria, quais logs/métricas AWS consultaria).
- Árvore de hipóteses (pelo menos 5 hipóteses para a causa raiz, ordenadas por probabilidade — considere que houve manutenção no TimescaleDB E mudança de security group).
- Para a hipótese mais provável: descrição detalhada de resolução, incluindo comandos.
- Ações pós-incidente: o que mudar para evitar recorrência (preventivo, não apenas detectivo).
- Comunicação: como comunicaria o status para os times afetados durante e após o incidente.

6. Estrutura de Entrega

Organize o repositório da seguinte forma (sugestão, adaptável):

```
trio-data-challenge/
├── README.md
├── docker-compose.yml
├── .env.example
├── init/
│   ├── timescaledb/
│   ├── postgres-legado/
│   └── clickhouse/
├── desafio-1/
│   ├── schemas/
│   ├── seed/
│   ├── queries/
│   ├── migration-analysis.md
│   └── REPORT.md
├── desafio-2/
│   ├── pipeline/
│   ├── diagrams/
│   └── ADR.md
├── desafio-3/
│   ├── backup/
│   ├── grafana/
│   ├── runbook.md
│   └── incident-response.md
└── docs/
```

6.1 Critérios de Aceitação Mínimos

- docker-compose up -d sobe todo o ambiente sem erros.
- Scripts de seed rodam e populam os bancos com volume relevante.
- Queries do Desafio 1 executam com sucesso e têm EXPLAIN ANALYZE documentado.

- Pelo menos um pipeline (TimescaleDB → ClickHouse) funciona end-to-end de forma demonstrável.
 - Pelo menos um dashboard Grafana está funcional consultando ClickHouse.
 - Documentação cobre decisões técnicas com justificativas.
 - Análise de migração PostgreSQL → Aurora está presente e fundamentada.
-

7. Apresentação Técnica

Após a entrega do repositório, agendaremos uma sessão de 30 a 45 minutos onde você irá:

4. Demonstrar o ambiente rodando (docker-compose up, dados carregados, queries executando, pipeline funcionando, dashboards no Grafana).
5. Apresentar as decisões arquiteturais e de modelagem, explicando trade-offs e alternativas consideradas.
6. Responder a perguntas técnicas de aprofundamento, como:
 - “E se o volume triplicasse, o que mudaria na arquitetura?”
 - “Como adicionaria um novo consumer no ClickHouse sem impactar as aplicações existentes?”
 - “Como faria a migração de engine de uma tabela ClickHouse em produção sem downtime?”
 - “Qual sua estratégia para onboardar um Data Champion novo que precisa criar queries no Grafana?”
 - “Se precisasse migrar o PostgreSQL legado para Aurora amanhã, qual seria seu plano de 72h?”
7. Discutir o cenário de incidente do Desafio 3, incluindo sua árvore de hipóteses e abordagem de resolução.

A apresentação não precisa de slides — preferimos ver terminal, código e dashboards ao vivo. Valorizamos a capacidade de comunicar decisões técnicas com clareza para audiência mista (engenharia + liderança).

8. Dúvidas e Contato

Se tiver dúvidas sobre escopo, interpretação de requisitos ou limitações técnicas do ambiente, entre em contato. Preferimos que você pergunte a que assuma incorretamente.

O que não é aceitável:

CONFIDENCIAL

- Cópia integral de soluções prontas sem adaptação ou compreensão.
- Uso indiscriminado de IA generativa sem revisão crítica — queremos ver seu raciocínio e experiência, não output de LLM. Na apresentação, faremos perguntas que exigem profundidade real.
- Entregar código que não funciona ou que você não sabe explicar.

O que valorizamos:

- Pragmatismo: soluções que funcionam, mesmo que não estejam perfeitas.
- Profundidade: preferível fazer 70% muito bem feito do que 100% superficialmente.
- Documentação como cidadã de primeira classe: se não está documentado, não existe.
- Ownership: trate como se fosse sua plataforma de produção.
- Visão cloud-native: pensar em como as soluções locais se traduzem para AWS em produção.

Boa sorte! Estamos ansiosos para ver como você pensa e constrói.
